Práctica 4. Gestión Avanzada de Ramas y Resolución de Conflictos

1. Contexto

Eres parte del equipo de desarrollo de **"WebNova"**, un proyecto web emergente. El equipo utiliza **Git** para gestionar versiones y desarrollar nuevas funcionalidades de forma paralela sin afectar la **rama principal** estable. Tu misión es simular el flujo de trabajo de un equipo, gestionando **ramas locales**, integrando funcionalidades y resolviendo **conflictos** de forma ordenada.

2. Objetivos

- Comprender la creación, navegación y gestión de ramas locales.
- Ejecutar fusiones (merges) limpias (Fast-Forward) y a tres bandas (3-Way Merge).
- Identificar, interpretar y resolver conflictos de fusión manualmente.
- Mantener un historial de proyecto estructurado y limpio.
- Representar visualmente la evolución del repositorio.

3. Preparación del Entorno (Fase Base)

1. Crea un directorio para el proyecto y accede a él:

   mkdir webnova
   cd webnova

2. Asegúrate de configurar la rama predeterminada como **main** e inicializa el repositorio:

   git config --global init.defaultBranch main
   git init

3. Crea el archivo base **index.html** con la estructura inicial:

   <h1>Bienvenido a WebNova</h1>
   Versión inicial del proyecto

4. Añade y confirma el archivo:

   git add .
   git commit -m "C1: Versión inicial de WebNova"

```
usuario@usuario-ALDA-PRO:~/ejercicios php$ cd ..
usuario@usuario-ALDA-PRO:~$ mkdir webnova
usuario@usuario-ALDA-PRO:~$ cd webnova
usuario@usuario-ALDA-PRO:~/webnova$ git config --global init.defaultBranch main
usuario@usuario-ALDA-PRO:~/webnova$ git init
Inicializado repositorio Git vacío en /home/usuario/webnova/.git/
usuario@usuario-ALDA-PRO:~/webnova$ nano index.html
usuario@usuario-ALDA-PRO:~/webnova$ git add .
usuario@usuario-ALDA-PRO:~/webnova$ git commit -m "C1: Versión inicial de WebNova"
[main (commit-raíz) 310ecaf] C1: Versión inicial de WebNova
 1 file changed, 3 insertions(+)
 create mode 100644 index.html
```

4. Desarrollo del Proyecto

PARTE 1: Desarrollo en Paralelo

1. **Creación de ramas de características:** Simularemos el desarrollo de tres componentes distintos creando tres ramas a partir de **main**:

   git branch cabecera
   git branch contenido
   git branch pie

2. Comprueba que las ramas se han creado y que te encuentras en **main**:

   git branch

3. **Desarrollo en la rama cabecera:** Cámbiate a la rama usando el comando moderno:

   git switch cabecera

4. Crea el archivo **header.html**:

   ```
   <header>
     <h2>WebNova - Innovación en la web</h2>
     <nav>
       <a href="#">Inicio</a> | <a href="#">Servicios</a> | <a href="#">Contacto</a>
     </nav>
   </header>
   ```

5. Añade y confirma:

   git add header.html
   git commit -m "C2: Añadido encabezado con navegación"

```
usuario@usuario-ALDA-PRO:~/webnova$ git branch cabecera
usuario@usuario-ALDA-PRO:~/webnova$ git branch contenido
usuario@usuario-ALDA-PRO:~/webnova$ git branch pie
usuario@usuario-ALDA-PRO:~/webnova$ git branch
  cabecera
  contenido
* main
  pie
usuario@usuario-ALDA-PRO:~/webnova$ git switch cabecera
Cambiado a rama 'cabecera'
usuario@usuario-ALDA-PRO:~/webnova$ nano header.html
usuario@usuario-ALDA-PRO:~/webnova$ git add header.html
usuario@usuario-ALDA-PRO:~/webnova$ git commit -m "C2: Añadido encabezado con navegación"
[cabecera 29e0106] C2: Añadido encabezado con navegación
 1 file changed, 6 insertions(+)
 create mode 100644 header.html
usuario@usuario-ALDA-PRO:~/webnova$
```

6. Desarrollo en la rama contenido:

git switch contenido

6. Crea el archivo **content.html**:

```
<main>
  <h3>Nuestros servicios</h3>
  <p>Ofrecemos soluciones de desarrollo web a medida.</p>
</main>
```

7. Confirma los cambios:

```
git add content.html
git commit -m "C3: Creado contenido principal"
```

```
create mode 100644 header.html
usuario@usuario-ALDA-PRO:~/webnova$ git switch contenido
Cambiado a rama 'contenido'
usuario@usuario-ALDA-PRO:~/webnova$ nano content.html
usuario@usuario-ALDA-PRO:~/webnova$ git add content.html
usuario@usuario-ALDA-PRO:~/webnova$ git commit -m "C3: Creado contenido principal"
[contenido 955a50f] C3: Creado contenido principal
 1 file changed, 4 insertions(+)
 create mode 100644 content.html
```

8. **Desarrollo en la rama pie:**

git switch pie

9. Crea el archivo **footer.html**:

```
<footer>
  <p>&copy; 2026 WebNova. Todos los derechos reservados.</p>
</footer>
```

10. Confirma los cambios:

```
git add footer.html
git commit -m "C4: Añadido pie de página"
```

```
usuario@usuario-ALDA-PRO:~/webnova$ git switch pie
Cambiado a rama 'pie'
usuario@usuario-ALDA-PRO:~/webnova$ nano footer.html
usuario@usuario-ALDA-PRO:~/webnova$ git add footer.html
usuario@usuario-ALDA-PRO:~/webnova$ git commit -m "C4: Añadido pie de página"
[pie 0a9749c] C4: Añadido pie de página
 1 file changed, 3 insertions(+)
 create mode 100644 footer.html
usuario@usuario-ALDA-PRO:~/webnova$
```

PARTE 2: Integración (Fusiones sin conflicto)

Es momento de ensamblar los componentes.

1. Vuelve a la rama principal:

   git switch main

2. Fusiona las ramas de desarrollo una a una. (Observa en la consola qué tipo de fusión realiza Git en cada caso):

   git merge cabecera
   git merge contenido
   git merge pie

3. Ahora que los tres archivos (**header.html, content.html, footer.html**) existen en la rama **main**, actualiza tu **index.html** para unificar el código (puedes copiar el contenido de los tres archivos dentro de **index.html**).
4. Haz un commit de consolidación:

   git commit -a -m "C5: Integrados cabecera, contenido y pie en index.html"

5. **Limpieza:** Elimina las tres ramas de desarrollo, ya que han cumplido su propósito:

   git branch -d cabecera contenido pie

```
usuario@usuario-ALDA-PRO:~/webnova$ git switch main
Cambiado a rama 'main'
usuario@usuario-ALDA-PRO:~/webnova$ git merge cabecera
Actualizando 310ecaf..29e0106
Fast-forward
 header.html | 6 ++++++
 1 file changed, 6 insertions(+)
 create mode 100644 header.html
usuario@usuario-ALDA-PRO:~/webnova$ git merge contenido
Merge made by the 'ort' strategy.
 content.html | 4 ++++
 1 file changed, 4 insertions(+)
 create mode 100644 content.html
usuario@usuario-ALDA-PRO:~/webnova$ git merge cabecera
git merge contenido
git merge pie
Ya está actualizado.
Ya está actualizado.
Merge made by the 'ort' strategy.
 footer.html | 3 +++
 1 file changed, 3 insertions(+)
 create mode 100644 footer.html
usuario@usuario-ALDA-PRO:~/webnova$ git commit -a -m "C5: Integrados cabecera, contenido y pie en index.html"
En la rama main
nada para hacer commit, el árbol de trabajo está limpio
usuario@usuario-ALDA-PRO:~/webnova$ git branch -d cabecera contenido pie
Eliminada la rama cabecera (era 29e0106).
Eliminada la rama contenido (era 955a50f).
Eliminada la rama pie (era 0a9749c).
usuario@usuario-ALDA-PRO:~/webnova$
```

PARTE 3: Provocando y Resolviendo un Conflicto

1. **La rama de rediseño:** Crea y salta inmediatamente a una nueva rama llamada **rediseño**:

   git switch -c rediseño

2. Modifica la etiqueta **<h1>** en **index.html** para que quede así:

   <h1>WebNova - Sitio oficial (Rediseño 2026)</h1>

3. Confirma el cambio:

   git commit -a -m "C6: Rediseño del título principal"

4. **Trabajo divergente en main:** Vuelve a la rama **main**:

   git switch main

5. Imagina que otro compañero modificó simultáneamente esa misma línea en **main**. Cambia el **<h1>** en **index.html** a:

   <h1>WebNova - Portal principal corporativo</h1>

6. Confirma el cambio:

   git commit -a -m "C7: Cambio de título corporativo en main"

7. **La Fusión y el Conflicto:** Intenta integrar el rediseño en la rama principal:

   git merge rediseño

8. Git pausará la fusión alertando de un conflicto de contenido.
9. **Resolución:** Abre **index.html** en tu editor. Identifica las marcas de conflicto **(<<<<<<<, =======, >>>>>>>).** Edita el archivo, elimina las marcas de Git y fusiona ambas ideas para dejar la línea final así:

   <h1>WebNova - Portal principal (Rediseño 2026)</h1>

10. Guarda, prepara el archivo y finaliza la fusión:

    git add index.html
    git commit -m "C8: Conflicto resuelto entre main y rediseño en el H1"

```
usuario@usuario-ALDA-PRO:~/webnova$ git switch -c rediseño
Cambiado a nueva rama 'rediseño'
usuario@usuario-ALDA-PRO:~/webnova$ nano index.html
usuario@usuario-ALDA-PRO:~/webnova$ git commit -a -m "C6: Rediseño del título principal"
[rediseño 63d97c1] C6: Rediseño del título principal
 1 file changed, 1 insertion(+), 3 deletions(-)
usuario@usuario-ALDA-PRO:~/webnova$ git switch main
Cambiado a rama 'main'
usuario@usuario-ALDA-PRO:~/webnova$ nano index.html
usuario@usuario-ALDA-PRO:~/webnova$ git commit -a -m "C7: Cambio de título corporativo en main"
[main 0189d24] C7: Cambio de título corporativo en main
 1 file changed, 1 insertion(+), 3 deletions(-)
usuario@usuario-ALDA-PRO:~/webnova$ git merge rediseño
Auto-fusionando index.html
CONFLICTO (contenido): Conflicto de fusión en index.html
Fusión automática falló; arregle los conflictos y luego realice un commit con el resultado.
usuario@usuario-ALDA-PRO:~/webnova$ nano index.html
usuario@usuario-ALDA-PRO:~/webnova$ git add index.html
usuario@usuario-ALDA-PRO:~/webnova$ git commit -m "C8: Conflicto resuelto entre main y rediseño en el H1"
[main f01686e] C8: Conflicto resuelto entre main y rediseño en el H1
```

PARTE 4: Visualización del Historial

Para comprobar que todas las ramas y fusiones se han registrado correctamente, ejecuta:

git log --oneline --graph --all

```
usuario@usuario-ALDA-PRO:~/webnova$ git log --oneline --graph --all
*   f01686e (HEAD -> main) C8: Conflicto resuelto entre main y rediseño en el H1
|\
| * 63d97c1 (rediseño) C6: Rediseño del título principal
* | 0189d24 C7: Cambio de título corporativo en main
|/
*   dc6e552 Merge branch 'pie'
|\
| * 0a9749c C4: Añadido pie de página
* |   63fc9ae Merge branch 'contenido'
|\ \
| * | 955a50f C3: Creado contenido principal
| |/
* / 29e0106 C2: Añadido encabezado con navegación
|/
* 310ecaf C1: Versión inicial de WebNova
usuario@usuario-ALDA-PRO:~/webnova$
```

5. Entregables

El alumno deberá entregar un documento PDF que incluya:

1. **Capturas de pantalla completas** de la terminal demostrando:
    - La resolución del conflicto (el estado con las marcas y los comandos de resolución).
    - La salida del comando **git log --oneline --graph --all** al finalizar la práctica.

2. **Reflexión teórica (Breve):**

    - ¿Por qué ocurrió el conflicto en la PARTE 3 y por qué no ocurrió en la PARTE 2?

      En a parte 3 ocurrió el conflicto porque la rama main co la rama rediseño modifico la misma linea exacta del index.html,por lo que git no pudo aplicar la versión, en la parte 2 no hubo conflictos porque la rama creo archivos por separado.

    - Al ejecutar **git switch main** en la PARTE 3.2, ¿qué ocurrió con los archivos de tu directorio de trabajo?

      El puntero se movió de la rama rediseño a la rama main ya que el directorio se actualizo de forma automática para asi saber el estado del ultimo commit en la rama main.